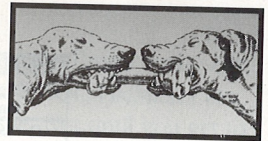


PROGRAMMER'S CHALLENGE

by Bob Boonstra, Westford, MA



Pente®

Reaching once again into the closet where we store board games, I found the game of Pente®, The board physically resembles the board used in GO, but the game strategies are simpler. Pente® is played by two players who alternate placing stones on a 19x19 grid. The objective is to win the game by getting five or more stones in a row or, alternatively, by capturing five or more pairs of your opponent's stones. Your Challenge is to write code that will play the game of Pente® and accumulate the most points (described below) in the minimum time.

The prototype for the code you should write is:

```
typedef struct Capture {
    Point stone1;
    Point stone2;
} Capture;

void InitPente(
    long boardHalfSize          /* e.g., 9 for a 19x19 board */
                                /* all coordinates between -boardHalfSize
                                and +boardHalfSize */
);

void Pente(
    Point opponentsMove,        /* your opponent moved here */
    Boolean playingFirst,       /* ignore opponentMove */
    Point *yourMove,            /* return your move here */
    Capture claimCaptures[],    /* return coordinates of captured pairs here */
    long *numCaptures,         /* return number of claimCaptures here */
    Boolean *claimVictory       /* return true if you claim victory with this move */
);

void TermPente(void);          /* deallocate any dynamic storage */
```

Captures take place by bracketing two adjacent stones of your opponents. Given the position

---BW---

... Black can capture the two White stones by playing ...

---BWB---

... after which the two White stones are removed ...

---B B---

Captures can occur horizontally, vertically, or diagonally. Note that no capture occurs if White moves into the unoccupied square below:

---BW-B---

Multiple captures can occur on a single move.

The game ends when one side captures five pairs of the opponent's stones, or when one side places five stones in a straight line, either horizontally, vertically, or diagonally. When one side obtains an unblocked row of four stones, called a tessera, a win is imminent. Therefore, an unblocked row of three stones, called a tria, is a serious threat that should be blocked unless a stronger offensive move exists.

To neutralize the advantage that the first player has, the first player's second move is restricted to be three or more intersections away from the center of the board (i.e., the h and v coordinates of White's second move must both be greater than or equal to 3 in absolute value).

At the start of the game, your `InitPente` routine (and that of your opponent) will be called with the half-width of the board in `boardHalfSize` (between 9 and 15, inclusive). The `Pente` routines will then be alternately called, providing the location of your `opponentsMove` (unless you are `playingFirst`). You should place a stone in an unoccupied location and return that location in `yourMove`. In addition, if your move captures any adjacent pairs of your opponent's stones, you should report the number of captures in `numCaptures`, and the locations of the captured pairs in `claimCaptures`. If your move results in victory, either by achieving 5 of your stones in a row, or a cumulative capture of 5 pairs of your opponent's stones, you should set `claimVictory` to `true`. At the end of the game, `TermPente` will be called, where you should deallocate any dynamically allocated storage.

THE RULES

Here's how it works: each month we present a new programming challenge. First, write some code that solves the challenge. Second, optimize your code (a lot). Then, submit your solution to MacTech Magazine. We choose a winner based on code correctness, speed, size, and elegance (in that order of importance) as well as the submission date. In the event of multiple equally desirable solutions, we'll choose one winner (with honorable mention, but no prize, given to the runner up). The prize for each month's best solution is a \$100 credit for Developer Depot™.

Unless stated otherwise in the problem statement, the following rules apply: All solutions must be in ANSI compatible C or C++, or in Pascal. We disqualify entries with any assembly in them (except for challenges specifically stating otherwise.) You may call any Macintosh Toolbox routine (e.g., it doesn't matter if you use `NewPtr` instead of `malloc`). We compile all entries into native PowerPC code with compiler options set to enable all available speed optimizations. The development environment to be used for selecting the winner will be stated in the problem. **Limit your code to 60 characters per line** or compress and binhex the

solution; this helps with e-mail gateways and page layout.

We publish the solution and winners for each month's Programmer's Challenge three months later. All submissions must be received by the 1st day of the month printed on the front cover of this issue.

You can get a head start on the Challenge by reading the Programmer's Challenge mailing list. It will be posted to the list on or before the 12th of the preceding month. To join, send an email to listserv@listmail.xplain.com with the subject "subscribe challenge-A".

Mark solutions "Attn: Programmer's Challenge Solution" and send it by e-mail to one of the Programmer's Challenge addresses in the "How to Communicate With Us" section on page 2 of this issue. Include the solution, all related files, and your contact info.

MacTech Magazine reserves the right to publish any solution entered in the Programmer's Challenge. Authors grant MacTech Magazine the exclusive right to publish entries without limitation upon submission of each entry. Authors retain copyrights for the code.

Board coordinates are expressed as distance from the center square in ordered (v,b) pairs, so that the center intersection is at (0,0), and the corners of the standard board are at (-9,-9), (-9,9), etc.

At the end of the game, points will be awarded as follows:

5 points for the player who achieved 5 stones in a row
1 point for each capture

1 point for each distinct sequence of 4 stones in a row remaining on the board

One point will be deducted for each second of execution time used during a player's turns, including the time taken by the InitPente and TermPente routines. Both the game winner and the game loser will accumulate points. It is possible to earn negative points. The Challenge winner will be the entry that accumulates the most points in a round-robin tournament of all entries, where each entry plays each other entry at least twice, half of the time playing first and half playing second.

Your code may allocate up to 10MB of dynamic storage, including both explicit calls to NewPtr/malloc and allocation of dynamic objects. This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions may be coded in C, C++, or Pascal.

Pente® is published by Pente Games, Inc.

THREE MONTHS AGO WINNER

Congratulations to Peter Lewis (Perth, Australia) for submitting the winning entry to the Stratego Challenge. Peter's entry was a convincing winner in a tournament of the 5 entries submitted.

The Stratego tournament consisted of 80 games, with each entry playing against each other entry 8 times, 4 times playing first and 4 times playing second. Peter's entry plays a very good game of Stratego, as evidenced by the fact that it won 30 of the 32 games it played. His algorithm, described in the commentary at the beginning of his code, includes aggressively attempting to capture weaker pieces, exploring with low ranking pieces of his own, running away from stronger pieces, exchanging pieces of equal rank, and advancing toward unknown pieces. Of the 30 games Peter's entry won, 10 were won by capturing the opponent's flag, and 19 were won by eliminating all of the pieces that the opponent could move. (The remaining win resulted from resignation of the opponent.)

Several of the entries used delaying tactics in an attempt to take advantage of the scoring rules and force their opponent to earn negative points for a win. As discussed on the Challenge mailing list, I cut off games where one player took longer than 20 seconds, declared a tie, and awarded points (sometimes negative points) to each player.

The table below lists the tournament results and point totals for each entry. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Code	Data	Language
Peter Lewis (37)	30	296.62	13436	310	C++
Dennis Jones	13	86.75	8632	414	C
Randy Boring (39)	5	58.68	5724	618	C
Ernst Munter (286)	9	29.45	12044	4072	C++
Tom Saxton (10)	7	-51.69	7320	460	C

TOP 20 CONTESTANTS

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	200	11.	Antoniewicz, Andy	24
2.	Gregg, Xan	63	12.	Picao, Miguel Cruz	21
3.	Lewis, Peter	57	13.	Day, Mark	20
4.	Cooper, Greg	54	14.	Higgins, Charles	20
5.	Boring, Randy	41	15.	Studer, Thomas	20
6.	Lengyel, Eric	40	16.	Saxton, Tom	17
7.	Mallett, Jeff	30	17.	Gundrum, Eric	15
8.	Murphy, ACC	30	18.	Hart, Alan	14
9.	Nicolle, Ludovic	28	19.	O'Connor, Turlough	14
10.	Larsson, Gustav	27	20.	Karsh, Bill	12

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points	5th place	2 points
2nd place	10 points	finding bug	2 points
3rd place	7 points	suggesting Challenge...	2 points
4th place.....	4 points		

Here is Peter's winning solution:

CHALLENGE.CP

© 1997 Peter N Lewis

```
#define ASSERTIONS 0
#define DEBUG_RULES 0
```

```
#include <Timer.h>
#include <stdlib.h>
#include <string.h>
```

```
#include "Challenge.h"
```

```
/*
```

Author: Peter N Lewis

Assumptions:

Only time we spend thinking is counted against out 10 seconds (not time in GetMove/ReportMove)

[Actually, this assumption is not valid, but Peter won anyway. —Bob]

Method:

Basically we keep track of the board and what we know and what they know. Each opponent piece has a bit map associated with it describing what pieces it could be. As we see more pieces, the bit map is culled. If the piece moves, the bomb & flag bits are removed. If we've seen all Scouts (for example), then the Scout bit is

PROGRAMMER'S CHALLENGE

by Bob Boonstra, Westford, MA

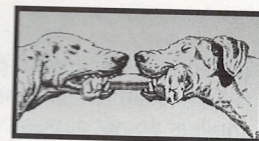


IMAGE DECODER

This month's Challenge is much easier to state than it will be to implement. Your task this month is to write a decoder for Graphics Interchange Format images. GIF is a data stream-oriented file format used to define the transmission protocol of LZW-encoded bitmap data. Your code will read a file containing a GIF and draw the image in an offscreen graphics world. There are two versions of the GIF specification; this Challenge includes only the earlier and more common GIF87a format, not the GIF89a format. The format description is too long to include here, but you can find the GIF87a specification at:

<<ftp://ftp.ncsa.uiuc.edu/misc/file.formats/graphics.formats/gif87a.doc>>

Other useful information on graphics formats in general is available at:

<<http://www.cis.ohio-state.edu/hypertext/faq/usenet/graphics/fileformats-faq/top.html>>

The prototype for the code you should write is:

```
#include <QDOffscreen.h>
#include <stdio.h>
GWorldPtr ReadImage(FILE *inputFile);
```

The `inputFile` will have been opened for you in binary mode by the calling routine. `ReadImage` should read and parse the image, create a `GWorld` with the appropriate color table, draw the image in the `GWorld`, and return a `GWorld` pointer to the calling routine. The solution that correctly decodes a variety of images in the least amount of execution time will be the winner.

I thought for some time about the Compuserve / Unisys / LZW patent controversy before selecting this topic for the Challenge. The LZW algorithm used in GIF compression is patented by Unisys, and Unisys requires that any commercial

software product that uses this algorithm, including software that reads GIF images, requires a license from Unisys. Freely distributed software does not require a license. Since the Challenge doesn't result in a software product, much less a commercial product, I decided to go ahead with this exploration of decoding efficiency.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions must be coded in C, C++, or Pascal. This Challenge was suggested by Peter Lewis, who earns 2 Challenge points for the suggestion.

THREE MONTHS AGO WINNER

Congratulations to **Jeff Mallett (Boulder Creek, CA)** for submitting the winning entry to the Pente™ Challenge. The objective was to accumulate, in minimum execution time, the most points in a round-robin tournament of the board game Pente™. With only three entries, the tournament required only six games for each player to compete against each other player twice, once playing first and once playing second. Both the winning entry and the third-place entry by Randy Boring employed an alpha-beta search technique to find the best move, and both entries were successful in winning most of their games. Jeff's entry, however, was more efficient about pruning the search tree and therefore required significantly less execution time than Randy's entry. Since a point was deducted for each second of execution time, the winning entry lost significantly fewer points for inefficiency. In addition, the scoring algorithm in Jeff's AddStone routine takes advantage of all three methods of scoring points: captures, winning by placing 5 stones in a row, and leaving sequences of 4-in-a-row on the board at the conclusion of a game. His solution earned more points as a result, even though it won the same number of games as Randy's.

THE RULES

Here's how it works: each month we present a new programming challenge. First, write some code that solves the challenge. Second, optimize your code (a lot). Then, submit your solution to MacTech Magazine. We choose a winner based on code correctness, speed, size, and elegance (in that order of importance) as well as the submission date. In the event of multiple equally desirable solutions, we'll choose one winner (with honorable mention, but no prize, given to the runner up). The prize for each month's best solution is a \$100 credit for Developer Depot™.

Unless stated otherwise in the problem statement, the following rules apply: All solutions must be in ANSI compatible C or C++, or in Pascal. We disqualify entries with any assembly in them (except for challenges specifically stating otherwise.) You may call any Macintosh Toolbox routine (e.g., it doesn't matter if you use `NewPtr` instead of `malloc`). We compile all entries into native PowerPC code with compiler options set to enable all available speed optimizations. The development environment to be used for selecting the winner will be stated in the problem. **Limit your code to 60 characters per line** or compress and binhex the

solution; this helps with e-mail gateways and page layout.

We publish the solution and winners for each month's Programmer's Challenge three months later. All submissions must be received by the 1st day of the month printed on the front cover of this issue.

You can get a head start on the Challenge by reading the Programmer's Challenge mailing list. It will be posted to the list on or before the 12th of the preceding month. To join, send an email to listserv@listmail.xplain.com with the subject "subscribe challenge-A".

Mark solutions "Attn: Programmer's Challenge Solution" and send it by e-mail to one of the Programmer's Challenge addresses in the "How to Communicate With Us" section on page 2 of this issue. Include the solution, all related files, and your contact info.

MacTech Magazine reserves the right to publish any solution entered in the Programmer's Challenge. Authors grant MacTech Magazine the exclusive right to publish entries without limitation upon submission of each entry. Authors retain copyrights for the code.

Spotlight Memory

NEW Debugger

Spotlight is the first automatic memory debugger for the Macintosh. Instantly detect invalid memory accesses, memory leaks, bad toolbox parameters, stack overwrites, memory relocation problems, and more. Spotlight can find problems in seconds that might take you a week to fix by hand.

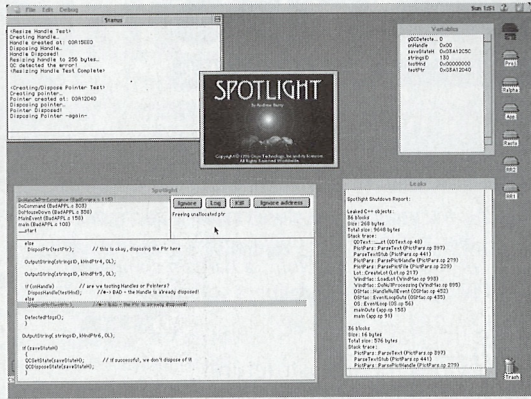
Spotlight uses your XSYM file to automatically patch your PowerPC native application. When Spotlight detects an error it brings up the exact source code line where the error occurred along with stack crawls, memory views, and program variable usage information.

No need to change your source code. No need to recompile. No learning curve whatsoever. Spotlight's Object Code Replacement instrumentation inspects each PowerPC processor memory read and write instruction in your code as it is run. Spotlight detects faulty memory reads and writes that occur between blocks, in released blocks, across multiple blocks, and outside of your heap.



Features

- Stops your program on exact source code line where a bad memory write occurs
- Complete leak reporting with source code stack trace showing point of allocation
- Fully validates parameters to over 400 Mac toolbox API calls.
- Works on all heap allocations Mac Toolbox, C 'malloc', C++ new and delete.
- Supports any language that produces an MPW or Metrowerks XSYM file.
- Full logging and ignore capabilities
- API for fine tuned control
- Purchase includes one year subscription to downloadable updates
- 30 day money back guarantee
- Available for secure electronic purchase now at our web site



Requirements

Macintosh or Mac OS-compatible computer with a PPC 601 or greater processor; RAM requirements vary depending on target application size; 2 MB hard disk space; System 7.5 or later; Metrowerks or MPW compatible XSYM symbolic file

Download free demo at <http://www.onyx-tech.com/>

\$199.00

TEL (941)795-7801 E-MAIL sales@onyx-tech.com
FAX (941) 795-5901 URL <http://www.onyx-tech.com/>

The table below lists the number of tournament wins, points earned, cumulative score, code size, data size, and programming language for each entry. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Score	Code	Data	Language
Jeff Mallett (74)	3	33	32.03	4872	7923	C
Sebastian Maurer	0	10	9.97	4860	208	C++
Randy Boring (66)	3	23	-374.90	7672	2649	C

TOP 20 CONTESTANTS

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants. The scores have been adjusted to correct an error made in assigning points for the Stratego Challenge printed in the November issue.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	200	11.	Higgins, Charles	20
2.	Boring, Randy	73	12.	Larsson, Gustav	20
3.	Cooper, Greg	61	13.	Studer, Thomas	20
4.	Lewis, Peter	59	14.	Gundrum, Eric	15
5.	Mallett, Jeff	50	15.	Hart, Alan	14
6.	Nicolle, Ludovic	48	16.	O'Connor, Turlough	14
7.	Murphy, ACC	34	17.	Picao, Miguel Cruz	14
8.	Gregg, Xan	33	18.	Saxton, Tom	12
9.	Antoniewicz, Andy	24	19.	Cutts, Kevin	11
10.	Day, Mark	20	20.	8-way tie	10

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points	5th place	2 points
2nd place	10 points	finding bug	2 points
3rd place	7 points	suggesting Challenge...	2 points
4th place.....	4 points		

Build great applications... Better Cheaper Faster!

Tools Plus

LIBRARIES + FRAMEWORK

Make cool apps while others are still reading their manuals. Professionally polished. Wickedly fast. Delightfully efficient.

"...the routines are more compact and faster than anything you might write... Every element of Tools Plus is useful... a bargain compared with coding these routines yourself."



You build the interface.

Tools Plus provides the infrastructure.

It makes all your pieces work together as an application.

With only a few hundred routines, Tools Plus thins your code by tens of thousands of lines. You see results sooner.

Changes are a snap.

"All in all, it's an incredibly rich collection of tools..."

If you are interested in developing applications that have 'quality' written all over them, then Tools Plus is for you."

MacTech MAGAZINE

Yes, Tools Plus has it!

- Create any element using a single routine
- Everything works as soon as you create it
- Automates all standard GUI elements
- Windows, Tool bar and floating palettes
- Buttons (all kinds, flat and 3D)
- Scroll bars (speed control, live scrolling)
- World-class custom controls
- Fields (w/scroll bars, filters, auto-editing)
- Picture buttons (the best anywhere)
- List boxes
- CDEF and LDEF automation
- Cursors (color, animated, auto-change)
- Menus (pull-down, hierarchical, pop-up)
- Edit menu (undo/redo, automatic editing)
- Panels (3D, group boxes, lines, panes)
- 3D titles (raised or inset)
- Extended multi-monitor and color support
- Clipboard automation
- Dynamic alerts (no resources required)
- Event processing automation
- Over 500K of custom fonts, icons, cursors, and other useful resources

Plus thousands more exciting features and services!

Results:

- ☒ Radical
- ☐ Slick
- ☒ Ravin*



FREE

SuperCDEFs™ \$89 value

- ✓ professionally designed and crafted controls
- ✓ dozens of 3D & flat buttons, tabs, sliders

Tools Plus for

Symantec (THINK) C/C++ (68K)	\$149
THINK Pascal (68K)	\$149
THINK C/C++ & THINK Pascal	\$199
CodeWarrior Bronze (68K)	\$199
CodeWarrior Gold (68K and PowerPC)	\$249

(We accept VISA and Amex only. Add \$10 for shipping.)

* Call for Academic pricing

Water's Edge Software
2441 Lakeshore Road West, Box 70022,
Oakville, Ontario Canada L6L 6M9

Orders and Enquiries:

Phone: (416) 219-5628
Fax: (905) 847-1638

WaterEdgSW@aol.com

Free Evaluation Kit:

Available at our web site
<http://www.interlog.com/~wateredg>



Here is Jeff's winning solution:

MyPente.c

Copyright © 1997 Jeff Mallett

```
//
// Do a depth=1 search to sort moves. Then do a higher
// depth alpha-beta search to select the best move.
//
// Plays for score rather than to win. For example,
// doesn't try to win by getting 5 captures since
// there is no score incentive for this. It could be
// retuned to play for a win instead.

#include "Pente.h"

#define SEARCH_DEPTH 4
enum {
    _FRIEND = 0x0001,
    _ENEMY = 0x0002,
    _EMPTY = 0x0004,
    _WALL = 0x0008
};

#define BORDER 1
#define BORDERS 2
#define INFINITY 32600
#define NA 0
#define MAX_CELLS (31 + BORDERS) * (31 + BORDERS)

#define ESTIMATE_PLUS 1
#define WIN 2000
#define CAPTURE_SCORE 240
#define VULNERABLE 150

static short CHAIN_SCORE2[3][3] = { //[color][open]
    { NA, NA, NA },
    { -1, -9, -2 }, // FRIEND
    { 1, 9, 2 } // ENEMY
};
static short CHAIN_SCORE3[3][3] = { //[color][open]
    { NA, NA, NA },
    { 3, 16, 40 }, // FRIEND
    { -2, -10, -30 } // ENEMY
};
static short CHAIN_SCORE4[3][3] = { //[color][open]
    { NA, NA, NA },
    { 230, 240, 250 }, // FRIEND
    { -150, -155, -160 } // ENEMY
};
static short CHAIN_SCORE5[3] = { // [color]
    NA, WIN, -WIN
};
//
// 1 2 3 4
static short BLOCK_SCORE[5] = { NA, 0, 14, 14, 22 };
static short THREATS[] = {
    NA, NA,
    25, // 2: tria
    30, // 3: half-open four
    350, // 4: double trias
    370, // 5: tria + half-open four
    450, // 6: tessera or 2 half-open fours
    500, 500, 500, 500, 500, 500, 500, 500, 500, 500
};

static short gPreScore[MAX_CELLS], gEstimates[MAX_CELLS];
static short *gEstimatesStart, *gEstimatesEnd;
static short gBoard[MAX_CELLS], *gBoardStart, *gBoardEnd;
static short *gFirstStone, *gLastStone;
static short *gKillers[SEARCH_DEPTH], gPreviousThreats;
static short gDirections[8], *gDirectionsEnd;
static short gMoveNum, gScore, gStartDepth;
static short gBoardHalfSize, gSideLength, gBoardMax;
static short gCumCapturesFriend, gCumCapturesEnemy;
static short gAdjustment, gSE;

#define ADJUST(a) ((a) + gAdjustment)
#define TRANSLATE(x, y) \
    (gSideLength * ADJUST(y) + ADJUST(x))
#define GET_INDEX(p) ((p) - gBoard)
#define GET_X(i) ((i) % gSideLength) - gAdjustment )
```

```

#define GET_Y(i)      ((i) / gSideLength) - gAdjustment )
#define UPDATE_ENDPOINTS(pSq, a, b) \
    if (pSq < a) a = pSq; \
    if (pSq > b) b = pSq

#define OPPONENT(side) (3 - (side))
#define OCCUPIED(x)    ((x) & (_FRIEND | _ENEMY))
#define EMPTY(x)      ((x) == _EMPTY)

// gChanges — Array of unsigned longs containing data to
// undo moves
// list of:
//   <pointer to position> <old position value>
// terminated by a 0L
static unsigned long gChanges[256], *gChangesEnd;
#define PUSH(x)          *(gChangesEnd++) = (x)
#define START_SAVE      PUSH(0L)
#define PUSH_SQ(pSq) \
    ( PUSH((long)*(pSq)); PUSH((unsigned long)(pSq)); )
#define POP              *--gChangesEnd
#define TOP              *gChangesEnd

static short * ChooseNextMove();
static short AddStone(short alpha, short beta, short *pSq,
    short color, short depth,
    short capturesFriend, short capturesEnemy,
    short *firstStone, short *lastStone);
static long MyFindCaptures(Capture capture[], short *pSq,
    short us, short them);
static Boolean MyFindFive(short *pSq, short color);

InitPente

void InitPente(
    long boardHalfSize /* e.g., 9 for a 19x19 board */
    /* all coordinates between -boardHalfSize
    and +boardHalfSize */
) {
    short i, j, *pSq;

    gBoardHalfSize = boardHalfSize;
    gSideLength = boardHalfSize * 2 + 1 + BORDERS;
    gSE = gSideLength + 1;
    gDirections[0] = -gSE; // NW
    gDirections[1] = -gSideLength; // N
    gDirections[2] = -gSideLength + 1; // NE
    gDirections[3] = -1; // W
    gDirections[4] = gSE; // SE
    gDirections[5] = gSideLength; // S
    gDirections[6] = gSideLength - 1; // SW
    gDirections[7] = 1; // E
    gDirectionsEnd = &gDirections[8];

    gBoardMax = gSideLength * gSideLength;
    i = (gSideLength + 1) * BORDER;
    gBoardStart = &gBoard[i];
    gBoardEnd = &gBoard[gBoardMax - i];
    gEstimatesStart = &gEstimates[i];
    gEstimatesEnd = &gEstimates[gBoardMax - i];
    gFirstStone = gBoardEnd;
    gLastStone = gBoardStart;

    pSq = gBoard;
    do {
        *pSq = _WALL;
    } while (++pSq != gBoardStart);
    do {
        *pSq = _EMPTY;
    } while (++pSq != gBoardEnd);
    do {
        *pSq = _WALL;
    } while (++pSq < &gBoard[gBoardMax]);
    for (i = BORDER + boardHalfSize * 2 + 1;
        i < gBoardMax - gSideLength; i += gSideLength)
        for (j = 0; j < BORDERS; ++j)
            gBoard[i + j] = _WALL;
    gCumCapturesFriend = gCumCapturesEnemy = gMoveNum = 0;
    gAdjustment = gBoardHalfSize + BORDER;
    gStartDepth = SEARCH_DEPTH - 1;
}

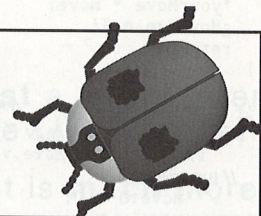
Pente

void Pente(
    Point opponentsMove, /* your opponent moved here */

```

TestTrack

Your Total Bug Tracking Solution

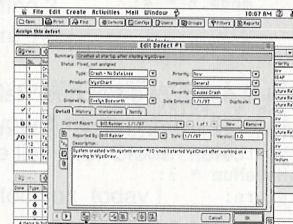


*"If you're a developer, you've got to get TestTrack.
Your bugs will hate you for it!"* Cool Tool of the Day

Discover the tool today's top software developers are using to improve the quality of their Macintosh and Windows applications—TestTrack.

- Track bugs, feature requests, problems, customer information, and more.
- New! E-mail notifications—SMTP and MAPI.
- New! Import bugs via e-mail automatically.
- Produce concise reports.
- Multiple users, full security—link engineers, testers, managers, even tech writers.
- New! Improved help desk support.
- New! Deferred defect numbering, external attachment storage, customer bug histories, and much, much, more.
- Automatically route bugs to team members.
- Track the history of each bug.
- Save time and improve tech support by giving Solo Bug, TestTrack's stand-alone bug reporter, to your customers.

Only \$169! 2+ for \$149 each!
To order call 888-683-6456
or 513-683-6456



Download our demo today!
<http://www.seapine.com>

sales@seapine.com
<http://www.seapine.com>

Seapine
Software

```

Boolean playingFirst, /* ignore opponentMove */
Point *yourMove, /* return your move here */
Capture claimCaptures[], /* return captured pairs here */
long *numCaptures, /* return # of claimCaptures */
Boolean *claimVictory /* return true if you claim
                        victory with this move */
) {
    Point move;
    Capture opponentCaptures[8];

    short *pSq, *pNewSq, *d;
    short score, bestScore, i;
    short *pBestMove;

    *numCaptures = 0;
    *claimVictory = false;
    if (playingFirst) { /**** MOVE 1
        move.h = move.v = 0;
        pSq = &gBoard[TRANSLATE(0, 0)];
        *pSq = _FRIEND;
        UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
        *yourMove = move;
        gMoveNum = 1;
        return;
    }

    i = TRANSLATE(opponentsMove.h, opponentsMove.v);
    pSq = &gBoard[i];
    *pSq = _ENEMY;
    UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
    ++gMoveNum;

    if (gMoveNum == 1) { /**** MOVE 2
        move.h = move.v = -2;
        pSq = &gBoard[TRANSLATE(-2, -2)];
        *pSq = _FRIEND;
        UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
    }
}

```

```

*yourMove = move;
gMoveNum = 2;
return;
}

if (gMoveNum == 2) { /** MOVE 3
    if (opponentsMove.v < opponentsMove.h) {
        if (opponentsMove.v < -opponentsMove.h) {
            // top triangle
            move.h = 0;
            move.v = 4;
        } else {
            // right triangle
            move.h = -4;
            move.v = 0;
        }
    } else {
        if (opponentsMove.v >= -opponentsMove.h) {
            // bottom triangle
            move.h = 0;
            move.v = -4;
        } else {
            // left triangle
            move.h = 4;
            move.v = 0;
        }
    }
    pSq = &gBoard[TRANSLATE(move.h, move.v)];
    *pSq = _FRIEND;
    UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
    *yourMove = move;
    gMoveNum = 3;
    return;
}

gChangesEnd = gChanges;
gCumCapturesEnemy +=
    MyFindCaptures(opponentCaptures, &gBoard[i],
        _ENEMY, _FRIEND);
for (pSq = gEstimatesStart; pSq != gEstimatesEnd; ++pSq)
    *pSq = 0;
for (pSq = gFirstStone; pSq <= gLastStone; ++pSq)
    if (OCCUPIED(*pSq)) {
        d = gDirections;
        do {
            pNewSq = pSq + *d;
            if (EMPTY(*pNewSq)) {
                gEstimates[GET_INDEX(pNewSq)] += ESTIMATE_PLUS;
                pNewSq += *d;
                if (EMPTY(*pNewSq)) {
                    gEstimates[GET_INDEX(pNewSq)] += ESTIMATE_PLUS;
                }
            }
        } while (++d != gDirectionsEnd);
    }

for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq)
    if (gEstimates[GET_INDEX(pSq)]) {
        i = GET_INDEX(pSq);
        gScore = gEstimates[i];
        gPreScore[i] = AddStone(-INFINITY, INFINITY, pSq,
            _FRIEND, 0, gCumCapturesFriend,
            gCumCapturesEnemy, gFirstStone,
            gLastStone);
    }

for (i=0; i<SEARCH_DEPTH; ++i)
    gKillers[i] = NULL;
bestScore = -INFINITY;
pBestMove = NULL;
pSq = ChooseNextMove();
while (pSq) {
    i = GET_INDEX(pSq);
    gScore = gEstimates[i];
    score = AddStone(-INFINITY, -bestScore, pSq, _FRIEND,
        gStartDepth, gCumCapturesFriend,
        gCumCapturesEnemy, gFirstStone, gLastStone);
    gEstimates[i] = -1; // searched
    if (score > bestScore) {
        bestScore = score;
        pBestMove = pSq;
    }
}

```

```

    pSq = ChooseNextMove();
}

if (bestScore == -INFINITY) {
    for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq)
        if (EMPTY(*pSq) && !gEstimates[GET_INDEX(pSq)]) {
            gScore = 0;
            score = AddStone(-INFINITY, -bestScore, pSq, _FRIEND,
                gStartDepth, gCumCapturesFriend,
                gCumCapturesEnemy, gFirstStone, gLastStone);
            if (score > bestScore) {
                bestScore = score;
                pBestMove = pSq;
            }
        }
    if (bestScore == -INFINITY) {
        DebugStr("\p no move found - Pente");
        return; //no move
    }
}

*pBestMove = _FRIEND;
UPDATE_ENDPOINTS(pBestMove, gFirstStone, gLastStone);
i = GET_INDEX(pBestMove);
move.h = GET_X(i);
move.v = GET_Y(i);
*yourMove = move;
++gMoveNum;
// find captures
*numCaptures = MyFindCaptures(claimCaptures, pBestMove,
    _FRIEND, _ENEMY);
gCumCapturesFriend += *numCaptures;
if (gCumCapturesFriend >= 5) {
    *claimVictory = true;
} else {
    *claimVictory = MyFindFive(pBestMove, _FRIEND);
}
}

void TermPente(void) { }

short * ChooseNextMove()
{
    short i, *pSq;
    short *found = NULL;
    short high = -INFINITY;
    for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq) {
        i = GET_INDEX(pSq);
        if (gEstimates[i] > 0 && gPreScore[i] > high) {
            found = pSq;
            high = gPreScore[i];
        }
    }
    return found;
}

```

```

// Alpha-beta search routine
// returns score of how good it is for color
// alpha and beta apply for the opponent after the move
// is made
// gScore is absolute; + for _FRIEND
short AddStone(short alpha, short beta, short *pSq,
    short color, short depth, short capturesFriend,
    short capturesEnemy, short *firstStone,
    short *lastStone)
{
    register short *pNewSq, *d;
    short x, bestScore, t, *pEnd;
    short open, *killer;
    short threats, blocks, vulnerable;
    short opponent = OPPONENT(color);
    short saveScore = gScore;

    START_SAVE;
    threats = blocks = vulnerable = 0;
    d = gDirections;
    do {
        pNewSq = pSq + *d;
        if (*pNewSq == color) { // Next to friend

```

```

if (d <= &gDirections[3]) {
    x = 1;
    open = 0;
    for (pNewSq += *d; *pNewSq == color; pNewSq += *d)
        ++x;
    if (EMPTY(*pNewSq))
        open = 1;
    for (pNewSq = pSq + *(d+4); *pNewSq == color;
        pNewSq += *(d+4))
        ++x;
    if (EMPTY(*pNewSq))
        ++open;
} else {
    t = *(pSq + *(d-4));
    if (t == color)
        continue;
    x = 1;
    open = 0;
    if (EMPTY(t))
        open = 1;
    for (pNewSq += *d; *pNewSq == color; pNewSq += *d)
        ++x;
    if (EMPTY(*pNewSq))
        ++open;
}

switch (x) {
    case 1: // 2-in-a-row
        gScore += CHAIN_SCORE2[color][open];
        if (open == 1)
            ++vulnerable;
        break;
    case 2: // 3-in-a-row
        gScore += CHAIN_SCORE3[color][open];
        if (open == 2)
            threats += 2; // tria = 2
        break;
    case 3: // 4-in-a-row
        gScore += CHAIN_SCORE4[color][open];
        threats += open * 3; // tessera = 6, half-open = 3
        break;
    default: // 5-in-a-row (or more)
        gScore += CHAIN_SCORE5[color];
        threats = -99; // game over
        break;
}

} else if (*pNewSq == opponent) { // Next to enemy
    x = 1;
    for (pNewSq += *d; *pNewSq == opponent; pNewSq += *d)
        ++x;

    if (EMPTY(*pNewSq)) {
        blocks += BLOCK_SCORE[x];
    } else if (x == 2 && *pNewSq == color) {
        t = CAPTURE_SCORE;
        if (color != _FRIEND)
            t = -t;
        gScore += t;
        if (color == _FRIEND) {
            if (++capturesFriend >= 5) {
                threats = -99; // game over
            }
        } else { // color == _ENEMY
            if (++capturesEnemy >= 5) {
                threats = -99; // game over
            }
        }
    }

    if (depth) {
        pNewSq = pSq + *d;
        PUSH_SQ(pNewSq);
        *pNewSq = _EMPTY;
        pNewSq += *d;
        PUSH_SQ(pNewSq);
        *pNewSq = _EMPTY;
    }
}

} while (++d != gDirectionsEnd);

if (threats < 0) {
    // Game over
    bestScore = gScore;
}

```

StoneTable

You thought it was **just** a replacement
for the List Manager ?

We lied, it is **much** more !

Tired of always adding just one more feature to your LDEF or
table code ? What do you need in your table ?

Pictures and Icons and Checkboxes ?
adjustable columns or rows ?
Titles for columns or rows ?
In-line editing of cell text ?
More than 32K of data ?
Color and styles ?
Sorting ?
More ??

How much longer does the list need to be to make it worth
\$200 of your time ?

See just how long the list is for StoneTable.

Make StoneTable part of your toolbox today !

Only \$200.00

MasterCard & Visa accepted.

StoneTable Publishing
More Info & demo Voice/FAX (503) 287-3424
<http://www.teleport.com/~stack> stack@teleport.com

```

if (color != _FRIEND)
    bestScore = -bestScore;
goto RESTORE;
}

// Not forced to make a bad move
if (color == _FRIEND) {
    if (gScore < saveScore)
        gScore = saveScore;
} else if (gScore > saveScore)
    gScore = saveScore;
if (depth) {
    // Add stone
    PUSH_SQ(pSq);
    *pSq = color;
    UPDATE_ENDPOINTS(pSq, firstStone, lastStone);
    -depth;
    gPreviousThreats = threats;
    bestScore = -INFINITY;
    // Killer move?
    killer = gKillers[depth];
    if (killer && EMPTY(*killer)) {
        saveScore = gScore;
        bestScore = AddStone(-beta, -alpha, killer,
            opponent, depth, capturesFriend,
            capturesEnemy, firstStone, lastStone);
        gScore = saveScore;
        if (bestScore > alpha) {
            if (bestScore >= beta) {
                bestScore = -bestScore;
                goto RESTORE;
            }
        }
        alpha = bestScore;
    }
}

pEnd = lastStone + gSE;
if (pEnd >= gBoardEnd)

```

```

    pEnd = gBoardEnd - 1;
    pSq = firstStone - gSE;
    if (pSq < gBoardStart)
        pSq = gBoardStart;
    do {
        if (EMPTY(*pSq) && pSq != killer) {
            d = gDirections;
            do {
                pNewSq = pSq + *d;
                if (OCCUPIED(*pNewSq) &&
                    (*pNewSq + *d) == *pNewSq ||
                    (*pSq - *d) == *pNewSq))
                    break;
            } while (++d != gDirectionsEnd);
            if (d != gDirectionsEnd) {
                saveScore = gScore;
                t = AddStone(-beta, -alpha, pSq, opponent,
                    depth, capturesFriend, capturesEnemy,
                    firstStone, lastStone);
                gScore = saveScore;
                if (t > bestScore) {
                    bestScore = t;
                    if (t > alpha) {
                        if (t >= beta) {
                            gKillers[depth] = pSq;
                            break;
                        }
                        if (depth >= gStartDepth-1)
                            gKillers[depth] = pSq;
                        alpha = t;
                    }
                }
            }
        } while (++pSq <= pEnd);
        if (bestScore == -INFINITY) {
            ++depth;
            goto TERMINAL;
        }
        bestScore = -bestScore;
    } else { // !depth
TERMINAL:
        bestScore = gScore;
        if (color != _FRIEND)
            bestScore = -bestScore;

//Winning threats
        if (gPreviousThreats >= 5) {
            bestScore -= THREATS[gPreviousThreats];
        } else if (threats >= 5) {
            bestScore += THREATS[threats];
        } else if (gPreviousThreats == 4) {
            bestScore -= THREATS[gPreviousThreats];
        } else if (threats == 4) {
            bestScore += THREATS[threats];
        } else if (gPreviousThreats) {
            bestScore -= THREATS[gPreviousThreats];
        } else if (threats) {
            bestScore += THREATS[threats];
        }
        bestScore += blocks - vulnerable * VULNERABLE;
    }

RESTORE:
    while (POP) {
        pSq = (short *)TOP;
        *pSq = POP;
    }

    return bestScore;
}

```

MyFindCaptures

```

long MyFindCaptures(Capture capture[], short *pSq,
    short us, short them)
{
    short i, *p1, *p2;
    short myCaptures = 0;
    short *d = gDirections;

    do {
        p1 = pSq + *d;

```

```

        if (*p1 == them) {
            p2 = p1 + *d;
            if (*p2 == them && *(p2 + *d) == us) {
                *p1 = *p2 = _EMPTY;
                i = GET_INDEX(p1);
                capture[myCaptures].stone1.h = GET_X(i);
                capture[myCaptures].stone1.v = GET_Y(i);
                i = GET_INDEX(p2);
                capture[myCaptures].stone2.h = GET_X(i);
                capture[myCaptures].stone2.v = GET_Y(i);
                ++myCaptures;
            }
        }
    } while (++d != gDirectionsEnd);
    return myCaptures;
}

```

Boolean MyFindFive(short *pSq, short color)

```

{
    short x, *d, *pNewSq, i;

    d = gDirections;
    i = 0;
    do {
        x = 1;
        for (pNewSq = pSq + *d; *pNewSq == color; pNewSq += *d)
            ++x;
        for (pNewSq = pSq + *(d+4); *pNewSq == color;
            pNewSq += *(d+4))
            ++x;
        if (x >= 5)
            return true;
        ++d;
    } while (++i < 4);
    return false;
}

```

MyFindFive

Want to know what products
are available for Mac OS
development? Check out

Developer
DEPOT™

<<http://www.devdepot.com>>